

宁波大学
NINGBO UNIVERSITY

信息安全技术实验报告

[Hash 函数加密算法实验]

姓名：	顾凯琪
学号：	226001624
课程名称：	信息安全技术
所在学院：	信息科学与工程学院
专业班级：	22 级计算机 2 班
任课教师：	徐博文 严迪群

实验项目名称	HASH 函数中 MD5、SHA-1 算法的加解密

一、实验目的与要求：

1. 熟悉 CrypTool 的基本功能；
2. 通过 CrypTool，完成 HASH 函数中的 MD5、SHA-1 加密算法的加解密过程。
3. 明文需要包含自己的学号或姓名；
4. 尝试用 Graphical Editor 实现 MD5、SHA-1 的加解密。

二、实验环境：

Windows10 系统
CrypTool 2.1 软件

三、实验方法（原理、流程图）

3.1.1 Hash 函数的概念

Hash 函数也称哈希函数/散列函数、杂凑函数，是一个从消息空间到像空间的不可逆映射，可将“任意”长度的输入经过变换以后得到固定长度的输出。它是一种单向密码体制，即只有加密过程，不存在解密过程。

Hash 函数的单向性和输出长度固定的特征使其可生成消息的“数字指纹”（Digital Fingerprint），也称消息摘要（MD,Message Digest）或哈希值/散列值（Hash Value），主要应用于消息认证、数字签名、口令的安全传输与存储、文件完整性校验等方面。

哈希值的生成过程表示为：h = H (M) h=H(M)h=H(M)，其中：

M 为任意长度的消息

H 为哈希(Hash)函数或称杂凑函数、散列函数

h 为固定长度的哈希值

3.1.2 Hash 函数的性质和结构

Hash 函数的一般结构称为迭代 Hash 函数结构，由 Merkle 和 amgård 分别独立提出。Hash 函数将输入消息分为 L LL 个固定长度的分组，每一分组长为 b bb 位，最后一个分组包含输入消息的总长度，若最后一个分组不足 b bb 位时，需要将其填充为 b bb 位。

Hash 函数输入消息为任意有限长度，输出哈希值是固定长度。请具有易计算、单向性、

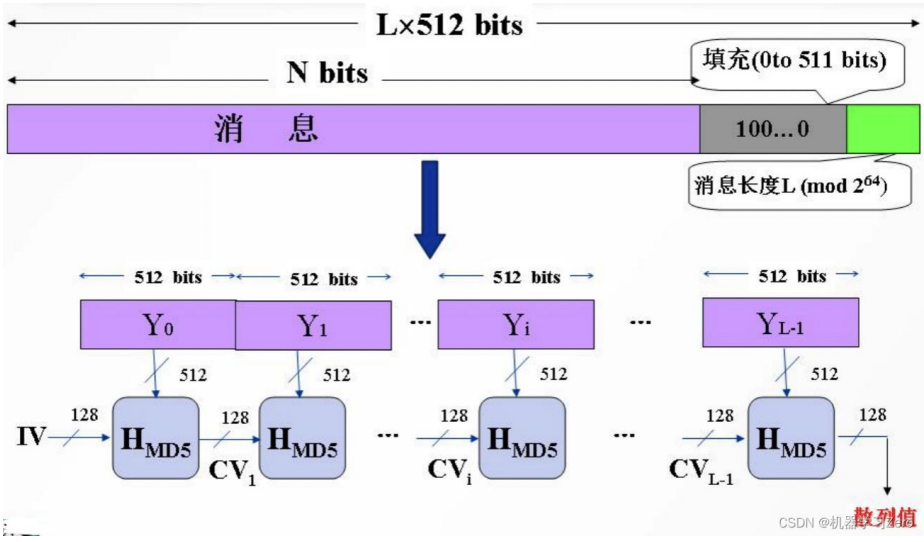
抗弱碰撞性以及抗强碰撞性。此外，Hash 函数应具有雪崩效应，即当消息的输入位发生变化时，输出的散列值至少有一半发生变化。

3.2 MD5 算法

MD5 算法由美国麻省理工学院著名密码学家 Rivest 设计，他于 1992 年向 IETF 提交的 RFC1321 中对 MD5 作了详尽的阐述。MD5 是在 MD2、MD3、MD4 的基础上发展而来，由于在 MD4 上增加了 Safety-Belts，MD5 又被称为是“系有安全带的 MD4”。

3.2.1 MD5 算法结构

MD5 算法的输入是最大长度小于 2^64 bit 的消息，输入消息以 512 bit 的分组为单位处



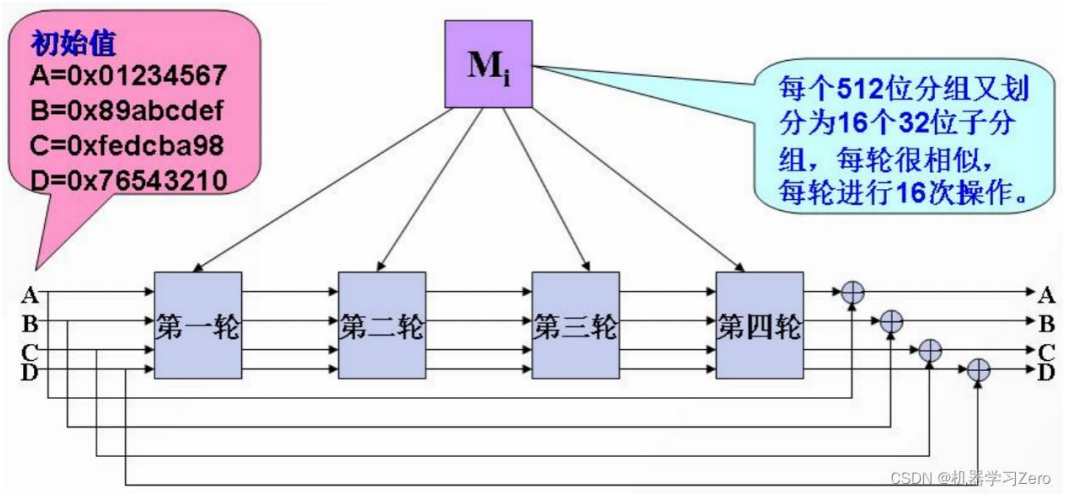
理，输出为 128bit 的消息摘要。

（1）附加填充位

填充一个“ 1 ”“1”“1”和若干个“ 0 ”“0”“0”使其长度模 512 与 448 同余，然后再将消息的真实长度以 64bit 表示附加在填充结果的后面，使得消息长度恰好为 512 bit 的整数倍，即 512 × L bit。

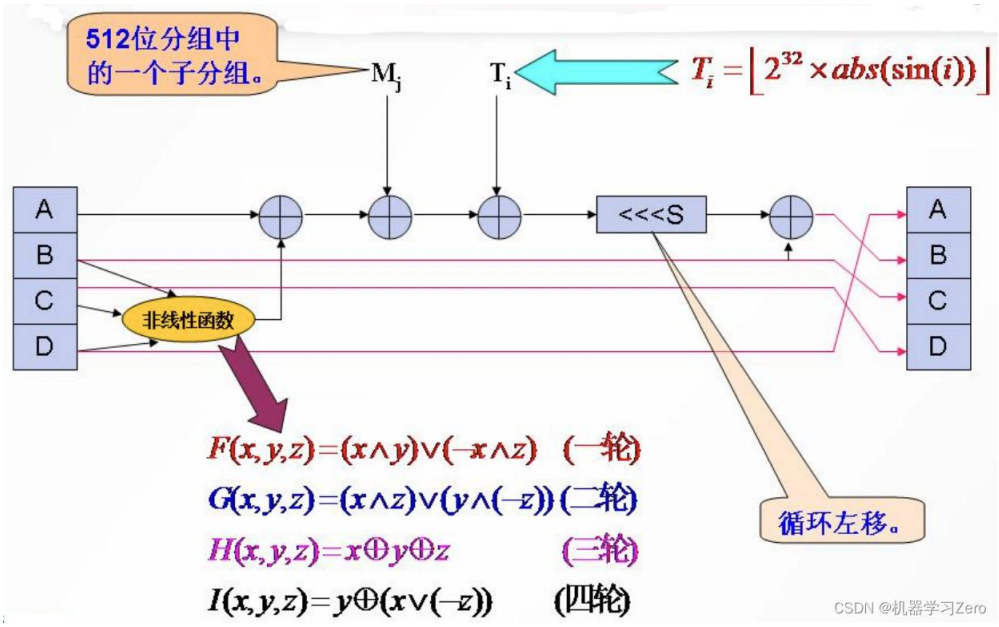
（2）分组处理(迭代压缩)

MD5 算法的分组处理(压缩函数)由 4 轮组成，512bit 的消息分组 M i 被均分为 16 个子分组(每个子分组 32bit)参与每 16 步函数运算。每步的输入是 4 个 32bit 的链接变量和一个 32bit 的消息子分组，输出为 32 位值。经过 4 轮共 64 步后,得到的 4 个寄存器值分别输入链接变量进行模加，即是当前消息的中间散列值。



3.2.2 压缩函数

MD5 的步函数，即压缩函数，先取向量(A , B , C , D) (A,B,C,D)(A,B,C,D)中的后 3 个作一次非线性函数运算，然后将所得的结果依次加上第 1 个变量、M [j]、T[i]，再将所得结果循环左移 s 位，并加上 (A,B,C,D)中的第 2 个变量 B，最后把新值赋给向量中的第 1 个变量。详细过程如下所示，其中 M[j]为消息分组 Mi 的第 j(0≤j≤15)个 32bit 子分组。



3.3 SHA1 算法

1993 年，美国国家标准技术研究所 NIST 公布了安全散列算法 SHA0（Secure Hash Algorithm）标准，1995 年 4 月 17 日，公布的修改版本称为 SHA-1，是数字签名标准中要求使用的算法。

2002 年，NIST 在 FIPS 180-1 的基础上发布了 FIPS 180-2，这个标准中除 SHA1 之外还新增加了 SHA256、SHA384 和 SHA512 三个散列算法标准。它们的消息摘要长度分别是 256 bit、384 bit 和 512 bit，以便与 AES 的使用相匹配。

3.2.1 算法结构

SHA1 算法的输入是最大长度小于 2^64bit 的消息，输入消息以 512 bit 的分组为单位处理，输出为 160bit 的消息摘要，因此抗穷举性更好。

SHA-1 设计基于 MD4，它有 5 个参与运算的 32 位寄存器，消息分组和填充方式与 MD5 相同，主循环也同样是 4 轮，但每轮进行 20 次操作，非线性运算、移位和加法运算也与 MD5 类似，但非线性函数、加法常数和循环左移操作的设计有一些区别。

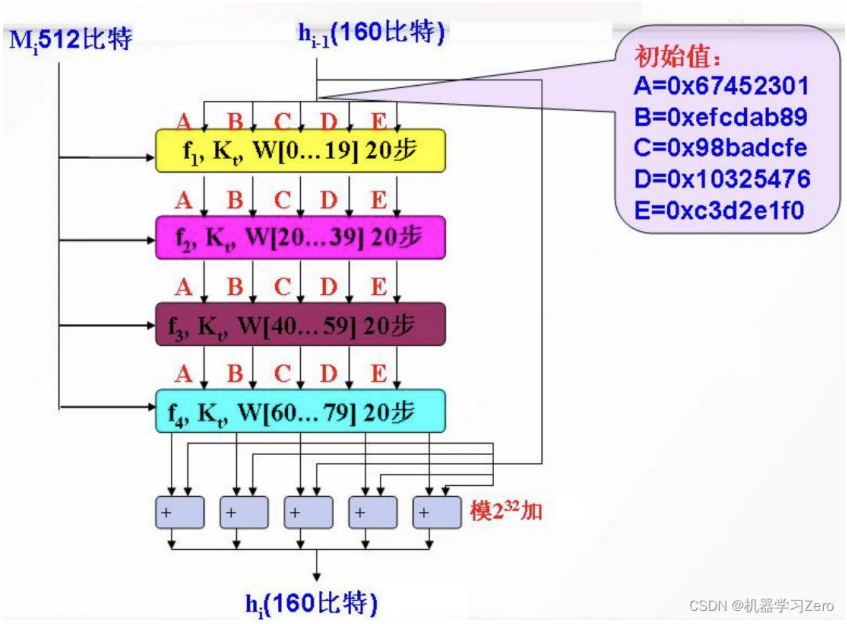
（1）附加填充位

填充一个 “1”和若干个 “0”使其长度模 512 与 448 同余，然后再将消息的真实长度以 64bit 表示附加在填充结果的后面，使得消息长度恰好为 512bit 的整数倍，即 512 × L bit。

（2）分组处理(迭代压缩)

SHA1 以 512 位的分组为单位处理消息，算法核心是一个包含 4 个循环的模块，每个环由 20 个步骤组成，每个循环使用的步函数相同，不同的循环中步函数包含不同的非线性函数(Ch、Parity、Maj、Parity)。

每一步函数的输入也不相同，除了寄存器 A 、 B 、 C 、 D A、B、C、DA、B、C、D 和 E EE 外，还有额外常数 K、与消息分组相关的 W[t]，其中 t (0 ≤ t ≤ 79) t(0≤t≤79)为步数。



每一循环均以当前正在处理的 512 512512 bit 的 Y q Y_qY q 和 160 160160bit 的缓存值 A、B、C、D 和 E 为输入，然后更新缓存的内容。最后一步的输模 2^{32}加上第一循环的输入 CV q 产生 CV_{q+1}。全部 512bit 数据块处理完毕后，输出 160bit 的 Hash 值。

3.2.2 压缩函数

SHA1 的步函数，即压缩函数每一轮循环的形式如下，其中 t(0≤t≤79)为步数。

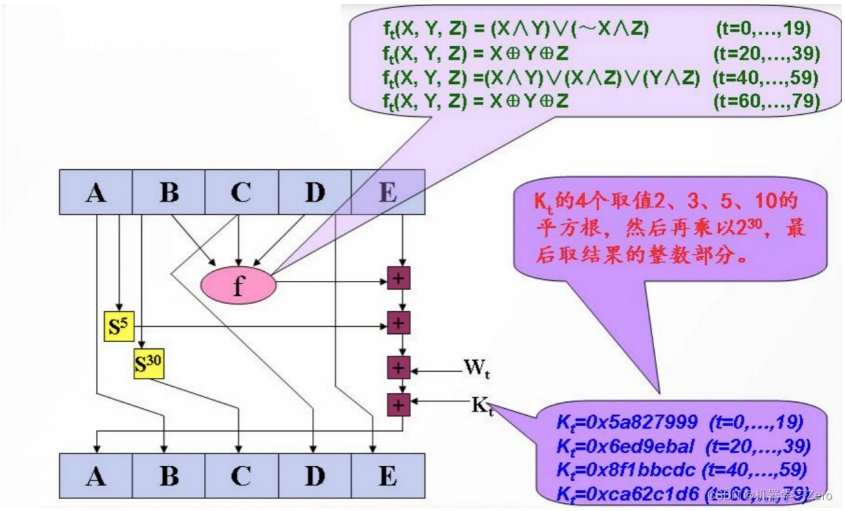
$$A = (ROTL^5(A) + f_t(B,C,D) + E + W_t + K_t) \bmod 2^{32}$$

$$B = A$$

$$C = ROTL^{30}(B) \bmod 2^{32}$$

$$D = C$$

$$E = D$$


$f_t(X, Y, Z) = (X \wedge Y) \vee (\sim X \wedge Z)$ (t=0,...,19)
$f_t(X, Y, Z) = X \oplus Y \oplus Z$ (t=20,...,39)
$f_t(X, Y, Z) = (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z)$ (t=40,...,59)
$f_t(X, Y, Z) = X \oplus Y \oplus Z$ (t=60,...,79)
A B C D E
f
S^5
S^{30}
W_t
K_t
A B C D E
K_t的4个取值2、3、5、10的平方根，然后再乘以2^{30}，最后取结果的整数部分。
K_t=0x5a827999 (t=0,...,19)
K_t=0x6ed9eba1 (t=20,...,39)
K_t=0x8f1bbcdc (t=40,...,59)
K_t=0xca62c1d6 (t=60,...,79)

四、实验过程、步骤及内容

本实验的主要目的是使用 Cryptool 和 Graphical Editor 对明文进行 MD5 和 SHA-1 加密，验证这两种哈希算法的基本特性。在实验的第一部分，启动 Cryptool 软件并进入 Hash 模块。首先设置输入明文为：“Liang Hangfeng 216001912 Learns Hash And MD5”，然后选择 MD5 算法进行计算。点击计算按钮后，系统会生成一个 160 位（20 字节）的 MD5 散列值。随后，通过稍微调整输入文本，例如改变其中的某些单词，重新计算哈希值，你会发现散列值发生了显著变化。这验证了 MD5 算法的敏感性，即使是微小的输入变动也会导致完全不同的哈希值。

在实验的第二部分，我们使用 Graphical Editor 进行 SHA-1 散列计算。启动 Graphical Editor 后，选择 SHA-1 模块并输入相同的文本：“Liang Hangfeng 216001912 Learns Hash And SHA-1”。点击计算按钮后，系统会生成一个 160 位（20 字节）的 SHA-1 散列值。这一部分验证了 SHA-1 算法同样具有不可逆性和对输入变化高度敏感的特点。输入明文的细微变化将导致散列值的巨大变化。

最后，在实验的第三部分，我们再次使用 Graphical Editor，但这次选择 MD5 算法来处理相同的输入文本。计算过程与前述步骤相似，最终得到一个 128 位（16 字节）的 MD5 散列值，进一步验证了 MD5 算法在处理明文时生成固定长度哈希值的特性。

通过本实验，我们深入理解了 MD5 和 SHA-1 算法的不可逆性，验证了它们对输入数据的变化极为敏感，且每次计算的结果都是唯一且不可还原的。

五、实验数据（现象）处理分析

该部分包含实验的主要操作过程与实验现象的记录。

5.1 使用 Cryptool 实现 MD5 处理

根据实验要求，使用 Cryptool 软件中的 Hash 模块中的相关函数进行 Hash 处理，对于 MD5，设置输入明文为“Gukaiqi226001624 Learns Hash And MD5”，如图 5.1 所示，处理得到的 Hash 结果为“6E 69 37 E0 2F 6A 3E B8 D0 5B 3C 35 B7 3D 45 2B”，总共 160 位（20 字节）。

同时稍微调整明文，得到的 Hash 值会有较大变化。

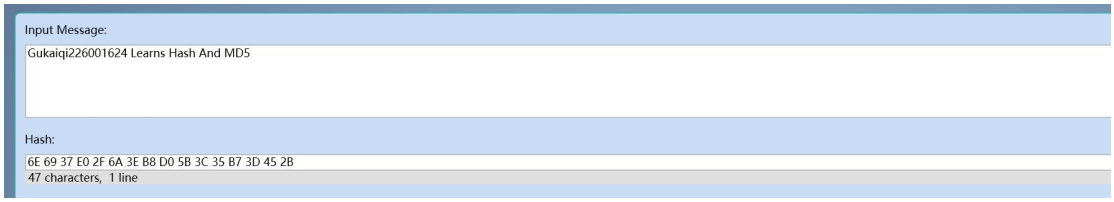


图 5.1 使用 Cryptool 进行 MD5 处理

对于 MD5，设置输入明文为“Gukaiqi226001624 Learns Hash And SHA-1”，如图 5.1 所示，处理得到的 Hash 结果为“57 92 C4 D6 C3 A0 DC 07 4B B6 BF 05 A1 D8 F8 08 B4 6D 47 6B”，总共 128 位（16 字节）。

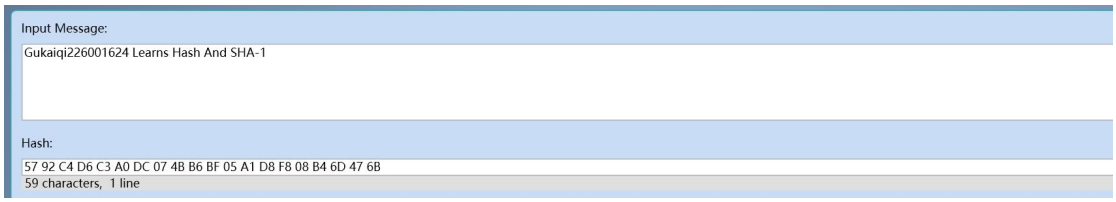


图 5.2 使用 Cryptool 进行 SHA-1 处理

对于上述 Hash 函数，均是不可逆的处理，因此无法从 Hash 值递推得到明文内容，即不存在解密环节。

5.2 基于 Graphical Editor 的 SHA-1 加密

使用 Graphical Editor 的相关组件进行 SHA-1 散列处理，结构较为简单，即包括基本的输入输出，与 SHA-1 模块，如图 5.3 所示。

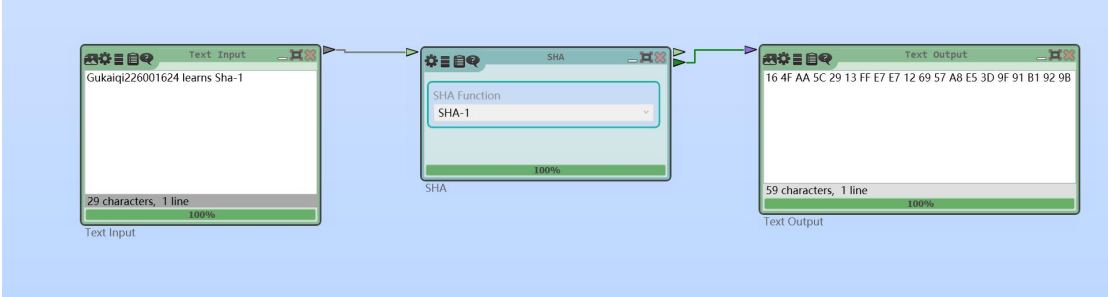


图 5.3 使用 Graphical Editor 进行 SHA-1 处理

从输出结果可以看到，SHA-1 的输出值为 20 字节的散列值，符合实验预期，且通过调整左侧 Text Input 的内容，Hash 结果有较大变化，输入与图 5.2 相同的数据，产生的 Hash 结果保持一致。

5.3 基于 Graphical Editor 的 MD5 加密

使用 Graphical Editor 的相关组件进行 MD5 散列处理，对于组件本身，Cryptools 提供了加密动画，内容与理论课程中相同，由于轮数较多，在本报告中不进行详细阐述，最终的加密结果如图 5.5 所示。可以看到，经过 MD5 的处理，最终原文被映射到一个长度为 16 字节的散列值上，符合实验预期。

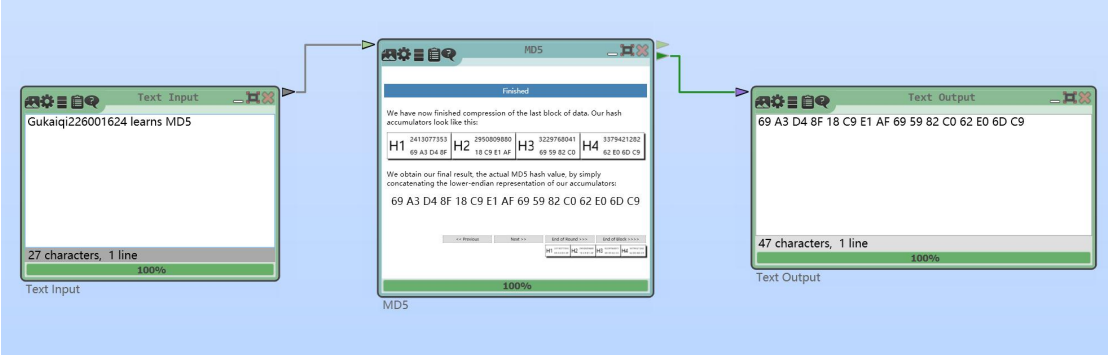


图 5.3 使用 Graphical Editor 进行 MD5 处理

六、实验结论

本次实验主要通过使用 CrypTool 和 Graphical Editor 工具，深入学习了 MD5 与 SHA-1 两种哈希算法的工作原理和应用。在实验过程中，我们验证了哈希函数的几个关键特性：

1. 哈希函数的单向性：实验明确展示了哈希函数的核心特性——单向性。我们从哈希值无法恢复原始明文，这一特性确保了数据在安全传输和存储时的不可逆性。
2. 哈希值的唯一性和抗碰撞性：即便是输入数据中的微小变化，也会导致哈希值显著变化，验证了哈希算法的抗碰撞性，即使是轻微的变动也能产生完全不同的结果。
3. 可视化理解算法：通过 Graphical Editor 的可视化工具，我们能够直观理解 MD5 与 SHA-1 算法的内部工作机制，使抽象的加密过程变得更加清晰易懂。
4. 算法的安全性考量：尽管 MD5 和 SHA-1 在现代密码学中已逐渐被认为不再足够安全，特别是在高安全要求的场景中，但它们依然在密码学基础和算法思想上具有重要意义。实验使我们理解了这些算法的设计背景与局限。
5. 实验技能的提升：通过实验，我们不仅掌握了 CrypTool 和 Graphical Editor 的使用技巧，还提高了对信息安全问题的分析和解决能力。
6. 信息安全的应用前景：哈希函数在信息安全领域具有广泛应用，包括消息认证、数字签名、密码存储与文件完整性校验等。本实验展示了哈希函数在保障信息安全中的基础作用。

综上所述，本次实验不仅增强了我们对 MD5 与 SHA-1 哈希算法的理论理解，也促使我们认识到在设计信息安全系统时，如何评估算法的安全性与应用场景。随着信息安全技术的不断发展，未来我们将更加关注更高效、更安全的加密与验证技术。

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：